

Intent-to-API Orchestration for Future Telco Networks: A CAPIF-Aware, AI-Native Platform Integrating CAMARA and NEF Services with Empirical Reliability Evaluation

Türkan Doğa Durak

Centre Tecnològic de Telecomunicacions de Catalunya (CTTC)

Castelldefels, Spain

turkandoga.durak@cttc.es

Abstract—Future Internet architectures demand programmable network exposure frameworks that can bridge high-level operator intents to concrete telecom API invocations without manual payload composition. This paper presents the design, implementation, and empirical evaluation of a cloud-native, intent-based telco orchestration platform combining 3GPP Common API Framework (CAPIF) integration, CAMARA-aligned service semantics, NEF-backed telecom realisation, and large language model (LLM)-assisted planning. The system is implemented as a multi-container FastAPI control plane with a product-style operator interface, live CAPIF invoker onboarding and service discovery, provider publication, persistent NEF data services, and a three-layer deterministic validator that guards all LLM-generated API payloads before execution. Both deterministic and LLM-assisted planning modes are supported; the latter grounded in runtime CAMARA OpenAPI schema injection (schema-grounded prompting) rather than vector-based retrieval. Empirical evaluation on a 60-entry bilingual (English/Turkish) ground-truth dataset using six formally defined reliability metrics yields: IWSR=100 % (deterministic) and IWSR=95 % (LLM-assisted, Claude Haiku4.5), with perfect Schema Compliance (SCR=100 %) and Semantic Validity (SVR=100 %) in both modes and a Stability Score of SS=0.938 across repeated LLM runs. All LLM failures are service-routing errors on deliberately ambiguous intents; no payload schema violation occurs in either mode. The platform and dataset are publicly available as open-source artefacts, providing a reproducible research baseline for intent-based orchestration in 5G and beyond-5G service exposure environments.

Index Terms—intent-based networking, CAPIF, CAMARA, NEF, 5G/B5G service exposure, LLM-assisted orchestration, cloud-native, future internet, reliability evaluation, zero-touch automation

I. INTRODUCTION

The emergence of Beyond-5G (B5G) and 6G network architectures elevates programmable service exposure from an optional feature to a foundational design principle of the Future Internet. Standardised frameworks such as 3GPP CAPIF (TS 23.222) [1] and the Linux Foundation CAMARA

project [2] define the governance, discovery, and semantic layers that enable vertical industries and developers to consume network capabilities through well-defined APIs. At the same time, NEF-based provider stacks [4] bridge those developer-facing interfaces to the underlying telecom data plane, resolving identifiers, enforcing policies, and returning structured monitoring results.

This two-tier model — CAPIF/CAMARA for exposure governance and NEF for realisation — creates an environment in which sophisticated orchestration logic is required: requests arrive as natural language intents from operators or automated management planes, and must be translated into precisely structured, multi-field API payloads complying with strict telecom domain constraints (E.164 phone formats, enumerated QoS profiles, duration bounds, CAPIF registry membership). Manual composition at this level is error-prone and scales poorly, motivating automated intent-to-API translation.

Large language models (LLMs) offer a compelling approach: their instruction-following capabilities can, in principle, translate free-text intents into structured API calls without explicit rule encoding [6], [8]. However, telecom API payloads impose hard correctness constraints that probabilistic generation cannot guarantee without external enforcement [7], [9]. Hallucinated service identifiers, malformed phone numbers, or invalid QoS profile names would silently propagate to production networks if not intercepted.

This paper addresses the gap between LLM planning capability and telecom-grade reliability through a platform that: (i) combines live CAPIF governance with multi-provider LLM planning; (ii) enforces correctness through a three-layer deterministic validator (schema, semantic policy, and CAPIF registry); (iii) provides a structured feedback correction loop for bounded iterative repair; and (iv) is evaluated on a bilingual 60-entry ground-truth dataset using six formally defined reliability metrics, yielding concrete, reproducible performance

figures.

Unlike conceptual architectures or simulation-only prototypes, the system reported here is fully deployed as a multi-container Docker stack, operates against live OpenCAPIF and live FRONT-research-group CAMARA providers, and includes a public open-source release of all code, datasets, and evaluation scripts.

The work reported here is grounded in three research questions:

- RQ1.** Can a schema-grounded two-phase LLM planner achieve telecom-grade payload correctness (SCR=SVR=100%) without fine-tuning?
- RQ2.** What is the primary failure mode of LLM-driven intent-to-API translation in a CAPIF-constrained service exposure environment?
- RQ3.** Does a bilingual (English/Turkish) intent set reveal statistically significant cross-lingual reliability differences?

The evaluation results answer all three questions empirically with quantified, reproducible measurements on an open dataset.

The remainder of the paper is structured as follows. Section II surveys related work. Section III states objectives and contributions. Section IV describes the platform architecture and cloud-native deployment. Section V details the backend orchestration design. Section VI covers the CAPIF integration workflow. Section VII describes CAMARA and NEF service realisation. Section VIII presents the LLM planning layer. Section IX defines the reliability evaluation framework and reports results. Section XI discusses findings, trade-offs, and limitations. Section XII concludes with future directions.

II. RELATED WORK

A. Intent-Based Networking and Orchestration

Intent-based networking (IBN) has been explored extensively at the NFV orchestration level. Rafiq et al. [10] demonstrated deterministic intent-to-descriptor translation across multiple orchestrators; Saraiva de Sousa et al. [12] identified intent abstraction as a key direction for adaptive service management; and Coronado et al. [11] positioned intent-driven closed-loop control as foundational to zero-touch 6G management. These works operate at the orchestration lifecycle level and generally assume structured intent inputs, not natural language.

B. 5G Network Exposure Infrastructure

On the exposure infrastructure side, Molina Sanchez et al. [3] implemented CAPIF Core Function as an open-source microservice (OpenCAPIF), enabling community experimentation with 3GPP-compliant API exposure. Fragkos et al. [4] introduced NEFSim as an open NEF experimentation platform covering the full 3GPP service exposure stack. Turkovic et al. [5] demonstrated 3GPP-compliant CAMARA QoD and Location API integration in a real CAM testbed, measuring end-to-end call-flow latency of 230–340 ms. Our work builds on these open platforms as provider components and focuses on the intent translation and orchestration layer above them.

C. LLMs for Telecom and Structured Output

Maatouk et al. [6] identified hallucination and output inconsistency as primary risks for LLM deployment in telecom operations. Liu et al. [9] benchmarked structured-output compliance of several leading LLMs and found that even GPT-4 achieves only an average compliance score of 0.40 on JSON schema tasks. Lee et al. [7] introduced TelAgentBench, confirming that multi-turn planning tasks significantly degrade LLM pass rates in telecom agent scenarios. Zeng et al. [8] demonstrated near-perfect workflow generation accuracy when LLMs are grounded in structured API descriptions — the approach adopted in Phase 2 of our planner.

D. Research Gap

To the best of our knowledge, no prior work provides end-to-end empirical reliability evaluation of LLM-based natural-language intent-to-API translation within a live CAPIF-compliant 5G service exposure environment, operating under simultaneous schema, semantic, and registry constraints, and reporting a standardised six-metric reliability framework on a bilingual ground-truth dataset.

III. SYSTEM OBJECTIVES AND CONTRIBUTIONS

The platform was built with four primary objectives:

- to unify CAPIF onboarding, discovery, and provider publication inside one operator-facing control plane,
- to expose CAMARA-aligned telecom capabilities through a live, guarded orchestration backend,
- to support both deterministic and LLM-assisted natural-language planning with explicit mode selection,
- to ensure that all execution paths remain guarded by deterministic validation and schema enforcement regardless of planning mode.

Its main contributions are:

- a working, openly-released CAPIF-aware telco orchestration backend operating against live CAMARA providers,
- a pluggable multi-provider LLM planning layer (Gemini, OpenAI, Anthropic) grounded by runtime CAMARA OpenAPI schema injection and live CAPIF discovery,
- a three-layer deterministic validator (L1 Pydantic schema, L2 telecom semantic policy, L3 CAPIF registry),
- a structured feedback correction loop for bounded iterative repair of invalid LLM-generated payloads,
- a bilingual 60-entry ground-truth evaluation dataset with difficulty stratification and multi-country phone number coverage,
- six formally defined reliability metrics (IWSR, SCR, SVR, HR, SS, RRF) constituting a reusable evaluation framework for intent-to-API orchestration systems.

IV. PLATFORM ARCHITECTURE AND CLOUD-NATIVE DEPLOYMENT

A. High-Level Component Model

The platform comprises nine functional components organised into three logical tiers: a user-facing *interface tier*,

a *control-plane tier* containing orchestration logic, and a *provider tier* containing telecom capability realisations.

TABLE I
IMPLEMENTED PLATFORM COMPONENTS

Component	Responsibility
Operator Interface	Product-style landing page with embedded OpenAPI console
FastAPI Backend	Route handling, orchestration control, validation, execution
Intent Engine	Deterministic keyword-based intent-to-step routing
LLM Planner	Two-phase structured planning with schema-grounded prompting
Multi-Layer Validator	L1 Pydantic schema, L2 semantic policy, L3 CAPIF registry
Feedback Engine	Bounded correction loop (max 3 iterations) for invalid payloads
CAPIF Client	Invoker onboarding, mTLS discovery, provider publication
Provider Layer	QoD, Location Retrieval, QoS Provisioning, QoS Profiles
Persistence Layer	MongoDB-backed NEF data with automatic initialisation

B. Cloud-Native Multi-Container Deployment

The platform is packaged as a Docker Compose stack (`docker-compose.capif.yml`) comprising five networked services:

- **backend** (FastAPI, port 8000): orchestration control plane, validator, LLM dispatcher, CAPIF client.
- **OpenCAPIF Core Function**: 3GPP-compliant register, discovery, and security services.
- **NEF provider** (port 8080): FRONT-research-group NEF stack exposing 3GPP Monitoring Event API.
- **camara-QoD provider** (port 8090): CAMARA Quality-on-Demand session management.
- **NEF-QoS provider** (port 8585): 3GPP AsSessionWithQoS adapter (TS 29.122).

All containers share a Docker bridge network; service discovery across providers is handled at run time by the CAPIF client querying the Core Function’s discovery endpoint rather than by static configuration. A named Docker volume persists MongoDB data across container restarts, and an initialisation script (`mongo-init.js`) seeds location, IMSI, and polygon records on first deployment.

The FRONT-research-group provider stacks are included as Git submodules, enabling version-tracked integration while keeping the main repository lightweight. Provider publication to CAPIF follows the standard APF sequence: onboard provider identity, submit OpenAPI metadata, and verify appearance in CAPIF discovery results.

C. Repository Structure

TABLE II
KEY REPOSITORY PATHS

Path	Role
<code>backend/main.py</code>	FastAPI entry point and route registration
<code>backend/intent_engine.py</code>	Deterministic planner and field extractor
<code>backend/llm_planner.py</code>	Two-phase LLM planning with schema grounding
<code>backend/validator.py</code>	Three-layer validation (L1/L2/L3)
<code>backend/feedback_engine.py</code>	Bounded correction loop
<code>backend/capif/client.py</code>	CAPIF onboarding, discovery, publish
<code>backend/evaluation_engine.py</code>	IWSR/SCR/SVR/HR/SS/RRF computation
<code>datasets/ground_truth.json</code>	60-entry bilingual evaluation dataset
<code>scripts/run_stability_test.py</code>	Stability Score (SS) measurement harness
<code>providers/ (submodules)</code>	4 FRONT-group CAMARA/NEF providers

D. Software Quality and Test Coverage

The platform includes 65 automated unit tests organised across three pytest modules: `test_intent_engine.py` (keyword routing coverage across all four services and edge cases), `test_validator.py` (L1/L2/L3 layer isolation tests, including boundary conditions for E.164 format, duration bounds, and QoS profile enumeration), and `test_executor.py` (orchestration flow tests in deterministic mode using fixture ground-truth entries). Tests are designed to be runnable without any LLM API key or live CAPIF connection, enabling CI/CD integration without external dependencies. The test suite serves as a regression guard for validator rules, ensuring that changes to schema definitions or semantic policy do not silently degrade correctness across the evaluation dataset.

V. BACKEND ORCHESTRATION DESIGN

A. Core API Surface

TABLE III
MAIN BACKEND ENDPOINTS

Method	Path	Purpose
GET	/health	Runtime health and mode summary
GET	/catalog/services	CAMARA-aligned capability catalog
POST	/orchestrate	Intent-to-execution planning
GET	/capif/status	CAPIF connectivity check
GET	/capif/invoke	Stored invoker identity
POST	/capif/invokers/onboard	Live invoker onboarding
GET	/capif/discover	Live CAPIF service discovery
POST	/capif/publish	Single service publication
POST	/capif/publish/catalog	Publish all catalog services

B. Supported Telecom Capabilities

The platform exposes four CAMARA-standardised orchestration targets: *Quality on Demand (QoD)* for temporary QoS session creation (CAMARA QoD v1.1.0); *QoS Profiles* for available profile enumeration (CAMARA QoS-Profiles v1.1.0); *QoS Provisioning* for long-lived QoS assignment via 3GPP AsSessionWithQoS (CAMARA QoS-Provisioning v1.1.0, TS 29.122); and *Location Retrieval* for device location access through NEF-backed MonitoringEvent subscriptions (CAMARA Location-Retrieval v0.5).

C. Planning Modes

POST /orchestrate accepts a mode field with three values. *Deterministic* mode applies keyword-based routing without any LLM call, providing a reliable, cost-free baseline always available. *LLM-assisted* mode invokes the two-phase planner and fails explicitly when the planning call returns an error, avoiding silent degradation. *Auto* mode attempts LLM planning first and falls back to deterministic mode on failure, enabling graceful degradation in production. This design keeps the AI layer optional while preserving full functionality regardless of LLM availability.

VI. CAPIF INTEGRATION WORKFLOW

A. CAPIF Roles and Governance Model

CAPIF provides the exposure governance plane for the entire platform. The backend acts as an **Invoker** consuming exposed APIs; CAMARA and NEF providers act as **API Exposing Functions (AEFs)** publishing service descriptions; and the OpenCAPIF stack provides the **Core Function** managing discovery, authentication, and onboarding [3]. This three-role model means that every API invocation is governed: services are visible to the orchestration layer only after publication, and the L3 validator can query the live registry to detect hallucinated service identifiers.

B. Invoker Onboarding Flow

The onboarding implementation in backend/capif/client.py follows the 3GPP CAPIF-2e reference point: (1) authenticate against the register service using pre-shared credentials; (2) generate an RSA private key and certificate signing request (CSR) locally; (3) submit the onboarding request to the CAPIF Core Function; (4) receive an apiInvokerId and invoker certificate chain; (5) store runtime credentials for subsequent mTLS-authenticated discovery and execution. The platform therefore operates as a real CAPIF invoker rather than a static mock client. CAPIF serves three distinct orchestration functions: **identity onboarding** (credential provisioning), **service governance** (API visibility after publication), and **discovery** (live registry queries at orchestration time).

C. Discovery and Publication

GET /capif/discover queries the CAPIF registry using the invoker's mTLS certificate and returns live service descriptions, including API names, endpoint URIs, and published security information. This discovery output is optionally injected into the LLM's Phase 1 prompt, grounding service selection in live registry state rather than static configuration. Provider publication follows the standard APF sequence: on-board provider identity, submit OpenAPI metadata through the CAPIF-4 reference point, and verify appearance in subsequent discovery responses.

VII. CAMARA AND NEF SERVICE REALISATION

CAMARA defines the API-facing service contracts (the *semantic layer*), while CAPIF provides the exposure governance layer. Provider services from the FRONT-research-group [5] bridge these interfaces to telecom behaviour. The QoD provider handles temporary session creation via the CAMARA QoD API, mapping requests to the underlying radio resource management plane. The NEF Monitoring Event provider resolves phone numbers to IMSI identifiers, reads location and polygon data from MongoDB, and returns a 3GPP-style monitoring result; the CamaraLocationRetrieval adapter then normalises this into the CAMARA Location Retrieval format [4].

The Location Retrieval end-to-end execution path comprises six steps: (1) intent mapped to location-retrieval service; (2) provider constructs a 3GPP MonitoringEvent subscription request; (3) provider obtains a CAPIF security token; (4) NEF API is called with the subscription; (5) NEF resolves the IMSI, reads polygon data from MongoDB; (6) response normalised to CAMARA format and returned to the caller.

VIII. LLM-ASSISTED PLANNING LAYER

A. Schema-Grounded Two-Phase Planning

backend/llm_planner.py implements a two-phase pipeline designed to separate service routing from payload generation, limiting each LLM call to a well-scoped subtask with a clearly defined schema target.

Phase 1 — Service Selection: The LLM receives (a) the user intent in natural language, (b) a schema-grounded summary of all CAMARA services and operations extracted at startup by parsing YAML OpenAPI descriptors, and (c) optionally, live CAPIF discovery results. It returns a JSON object specifying `service_id`, `operation_id`, `method`, `path`, and a `rationale` field. The rationale enables downstream traceability without additional instrumentation.

Phase 2 — Payload Generation: Given the Phase 1 selection, the LLM receives the full CAMARA OpenAPI schema for that specific operation, extracted by parsing the appropriate YAML file at runtime. It returns a complete, schema-aligned JSON payload matching the operation’s request body specification. Grounding the payload prompt in the actual operation schema (rather than a generic description) is the key mechanism achieving SCR=100% in LLM-assisted mode.

The planner proposes structured actions but never executes them directly; all output passes through the three-layer validator before any provider call is made.

B. Multi-Provider LLM Dispatcher

Three LLM providers are supported via a unified dispatcher interface: Gemini (GEMINI_API_KEY), OpenAI (OPENAI_API_KEY), and Anthropic (ANTHROPIC_API_KEY). The dispatcher selects the active provider from environment configuration, enabling provider substitution without code changes. The full 60-entry evaluation in this paper used **Claude Haiku 4.5** (model: `claude-haiku-4-5-20251001`, temperature 0.1) at a total API cost of approximately \$0.35 across all evaluation and stability-score runs (300 total LLM call pairs), demonstrating that empirical reliability measurement is accessible even at very low budget.

C. Structured Feedback Correction Loop

When Phase 2 output fails the three-layer validator, the feedback engine constructs a structured correction prompt containing: the original intent, the rejected payload JSON, and layer-tagged error messages (e.g., `[L1] device.phoneNumber must match E.164`). This prompt is submitted to the same LLM for regeneration. The loop repeats up to `MAX_ITERATIONS=3`, after which the most recent attempt is returned with the validation error attached. Recovery metadata is recorded in a `FeedbackMetadata` object and included in the orchestration response, enabling offline analysis of correction convergence rates.

D. Representative Orchestration Example

An LLM-assisted dry-run request with intent “Boost video QoS for +905551112233 for 15 minutes” produced: `service=qod`, `operation=createSession`, `path=/quality-on-demand/v1/sessions`, `duration=900s`, `profile=QOS_E_STREAMING`. The three-layer validator confirmed structural, semantic, and registry compliance in a single pass, and no feedback iteration was required.

IX. RELIABILITY EVALUATION FRAMEWORK

A. Formal Metric Definitions

Intent-to-API translation is formalised as a mapping $f_\theta : \mathcal{I} \rightarrow \mathcal{W}$, where $\mathcal{I}$ is the set of natural language intents and $\mathcal{W}$ is the set of CAPIF-compliant API workflows generated by LLM f_θ (or the deterministic engine in baseline mode).

1) *Intent-to-Workflow Success Rate (IWSR):*

$$\text{IWSR} = \frac{|\{w \in \mathcal{W} \mid w = w^*\}|}{|\mathcal{I}|} \quad (1)$$

where w^* is the ground-truth workflow. IWSR is the primary end-to-end correctness metric: it requires both correct service selection *and* a valid, semantically correct payload matching the ground truth.

2) *Schema Compliance Rate (SCR):*

$$\text{SCR} = \frac{|\{w \in \mathcal{W} \mid w \models \text{Schema}\}|}{|\mathcal{W}|} \quad (2)$$

Fraction of generated payloads passing Layer 1 (Pydantic) structural validation, independent of semantic correctness.

3) *Semantic Validity Rate (SVR):*

$$\text{SVR} = \frac{|\{w \in \mathcal{W} \mid w \models \text{TelecomPolicy}\}|}{|\{w \in \mathcal{W} \mid w \models \text{Schema}\}|} \quad (3)$$

Among schema-compliant payloads, the fraction that also pass Layer 2 telecom domain policy checks: E.164 phone format, valid QoS profile identifiers, duration bounds (1–7200 s), IPv4 unicast address validation, and operation/method consistency.

4) *Hallucination Rate (HR):*

$$\text{HR} = \frac{|\{w \in \mathcal{W} \mid w \not\subseteq \mathcal{A}_{\text{reg}}\}|}{|\mathcal{W}|} \quad (4)$$

where $\mathcal{A}_{\text{reg}}$ is the set of APIs present in the CAPIF live registry. HR measures service-selection hallucinations: the LLM returns a service identifier that does not exist in the registered catalog.

5) *Stability Score (SS):* Given k repeated runs of the same intent set, each producing a workflow, with d distinct output tuples observed across runs:

$$\text{SS} = 1 - \frac{d-1}{k} \quad (5)$$

Perfect stability ($d = 1$, identical output every run) yields $\text{SS} = 1.0$. SS is computed by running the 60-entry dataset $k = 5$ times and measuring output variance per entry averaged over all entries.

6) *Recovery Rate after Feedback (RRF):*

$$\text{RRF} = \frac{|\{w_c \mid w_i \not\models \text{Schema} \wedge w_c = w^*\}|}{|\{w_i \mid w_i \not\models \text{Schema}\}|} \quad (6)$$

Among initially schema-invalid payloads, the fraction that the feedback loop successfully corrects to a ground-truth-matching workflow. RRF is undefined (denominator=0) when $\text{SCR} = 100\%$, i.e., when no payload fails schema validation on the first attempt.

B. Evaluation Dataset

The ground-truth dataset (`datasets/ground_truth.json`) contains 60 manually crafted entries spanning all four supported services: 20 for QoD Session creation, 15 for Location Retrieval, 10 for QoS Profile discovery, 10 for QoS Provisioning, and 5 deliberate edge cases (ambiguous or boundary-condition intents). Difficulty is stratified as: 24 easy, 20 medium, 16 hard. Language coverage is 32 English and 28 Turkish entries, providing bilingual robustness evaluation without additional prompt engineering. Phone numbers span four country codes (+30 GR, +90 TR, +34 ES, +1 US) sourced from FRONT-research-group test fixtures, testing E.164 format validation across international prefixes. The dataset and evaluation scripts are released alongside the platform.

C. Evaluation Results

TABLE IV
RELIABILITY METRICS: DETERMINISTIC VS. LLM-ASSISTED MODE (60 ENTRIES)

Metric	Deterministic	LLM (Claude Haiku 4.5)
IWSR	100.0 %	95.0 %
SCR	100.0 %	100.0 %
SVR	100.0 %	100.0 %
HR	0.0 %	5.0 %
SS	1.000	0.938
RRF	N/A	N/A ^a
Avg. latency	<1 ms	≈3 s

^aRRF is undefined when all payloads pass schema validation on first attempt (SCR = 100%; the feedback loop was never triggered).

TABLE V
IWSR BREAKDOWN BY DIMENSION (LLM-ASSISTED MODE)

Dim.	Category	N	IWSR
Language	English	32	93.8 %
	Turkish	28	96.4 %
Difficulty	Easy	24	100.0 %
	Medium	20	100.0 %
	Hard	16	81.2 %
Service	QoD Session	20	90.0 %
	Location Retrieval	15	100.0 %
	QoS Profiles	10	100.0 %
	QoS Provisioning	10	100.0 %
	Edge Cases	5	80.0 %

D. Error Analysis

The 5 % IWSR gap in LLM mode corresponds to 3 entries out of 60, all classified as service-selection errors (HR = 5%): the LLM selects a valid, registered service but the wrong one for the given intent. Two failure patterns were identified.

Pattern 1 — QoD vs. QoS Provisioning ambiguity (2 entries): Intents containing “critical communications QoS” or “emergency QoS” caused the model to select

qos-provisioning (long-lived permanent assignment) instead of qod (temporary session). The model correctly identifies the QoS priority requirement but misinterprets the temporal scope from the phrasing.

Pattern 2 — Edge case boundary (1 entry): An intent specifying a very short duration (“QoS for 30 seconds”) caused route confusion between a QoD temporary session and a provisioning operation. Explicit temporal quantifiers in the future prompt design would address both patterns.

Critically, in all failure cases, the *payload* generated for the wrongly-selected service was structurally and semantically valid (SCR = SVR = 100%), confirming that Phase 2 schema grounding fully prevents payload-level errors; the residual failure mode is entirely in Phase 1 service routing.

X. PERSISTENCE AND THREE-LAYER VALIDATION

A. Persistent NEF Data

The NEF stack stores location and polygon records in MongoDB. A named Docker volume at `/data/db` prevents data loss across container restarts, and an auto-seeding initialisation script at `/docker-entrypoint-initdb.d/mongo-init.js` populates required records on first deployment. Subsequent restarts preserve the existing dataset without manual reseeding, enabling reproducible evaluation across sessions.

B. Three-Layer Validation Detail

Execution never depends solely on LLM output. All generated payloads pass three independent validation tiers sequentially before any provider call is made:

- L1 — Schema (SCR):** Pydantic v2 model validation enforcing required fields, correct types, nested object structure, and field constraints (e.g., duration as positive integer). Failures produce structured error objects returned to the feedback engine.
- L2 — Semantic Policy (SVR):** Eight domain-specific deterministic rules: E.164 phone number format (`^[+][1-9]\d{7,14}$`), valid QoS profile identifiers from the CAMARA enumeration, duration within 1–7200 s, IPv4 unicast address validation, operation/method consistency, and device field completeness.
- L3 — CAPIF Registry (HR):** Live CAPIF discovery query to verify the selected `service_id` exists in the published registry. This layer is non-blocking on CAPIF unavailability (fails open) but records the check result in the response for audit traceability.

This architectural separation between probabilistic LLM reasoning and deterministic telecom constraint enforcement is the central property enabling quantified reliability guarantees independent of LLM quality.

XI. DISCUSSION

A. Comparison with Related Work

Table VI positions this work against the closest related efforts along five dimensions relevant to Future Internet service exposure.

TABLE VI
COMPARISON WITH RELATED WORK (✓ = YES; × = NO)

Work	CAPIF	LLM	Valid.	Metrics
Rafiq et al. [10]	×	×	×	None
Molina et al. [3]	✓	×	×	None
Fragkos et al. [4]	×	×	×	None
Turkovic et al. [5]	✓	×	×	Latency
Maatouk et al. [6]	×	✓	×	Qual. only
This work	✓	✓	✓	IWSR, SCR, SVR, HR, SS

No prior work combines all five dimensions simultaneously. The absence of live CAPIF integration in most LLM-for-telecom studies means that hallucinated service identifiers (measured by HR) cannot even be detected, let alone counted. Our platform is unique in making all five properties co-present and empirically evaluated.

B. Insight: Schema Grounding Solves Payload Errors

The most significant empirical finding is that schema-grounded Phase 2 prompting eliminates payload-level LLM errors entirely (SCR = SVR = 100%). This confirms the Flow-Mind hypothesis [8] in a live telecom environment: providing the exact operation schema as context is sufficient for an instruction-following LLM to generate structurally and semantically valid API payloads. The remaining 5% failure is a service-routing problem (Phase 1), which is a coarser-grained classification task amenable to improved prompting, few-shot examples, or fine-tuning.

C. Deterministic vs. LLM: Complementarity, Not Competition

The deterministic mode achieves IWSR = 100% but requires explicit keyword matching for every intent pattern, making it brittle to novel phrasing and non-English inputs. LLM-assisted mode achieves IWSR = 95% with zero implementation cost per new intent pattern and robust cross-lingual coverage (Turkish = 96.4%, English = 93.8%). The *auto* mode combining both — LLM primary, deterministic fallback — offers the best practical trade-off: near-LLM flexibility with deterministic reliability as a safety net.

D. Cost and Scalability

At \$0.35 for 300 two-phase LLM call pairs (60 entries × 5 stability runs), the per-intent cost is approximately \$0.0012 at Claude Haiku 4.5 pricing. The three-layer validator adds sub-millisecond overhead, making it negligible relative to network round-trips. The platform’s stateless control plane scales horizontally without architectural changes; the main scalability bottleneck at higher loads would be LLM API rate limits, addressable through request batching or model serving infrastructure.

E. Limitations

The current evaluation has three notable limitations. First, only one LLM (Claude Haiku 4.5) is benchmarked; cross-model comparison is deferred to future work. Second, the

60-entry dataset, while bilingual and difficulty-stratified, is manually crafted; ecological validity against real operator intent logs remains unverified. Third, execution is performed in *dry-run* mode during evaluation; end-to-end live execution success rates may differ from IWSR due to transient provider or network errors not modelled in the current framework.

XII. CONCLUSION

This paper presented a cloud-native, AI-native telco orchestration platform for intent-to-API planning in Future Internet service exposure environments. The system integrates 3GPP CAPIF governance, CAMARA service semantics, NEF-backed telecom realisation, three-layer deterministic validation, a structured feedback correction loop, and multi-provider LLM-assisted planning into one deployable, open-source control plane.

Empirical evaluation on a bilingual 60-entry dataset using six formally defined reliability metrics demonstrates that the combination of schema-grounded two-phase LLM prompting and deterministic validation achieves: IWSR = 95% (LLM-assisted), IWSR = 100% (deterministic), SCR = SVR = 100% in both modes, SS = 0.938, and zero payload schema violations. All residual LLM failures are service-routing errors on deliberately ambiguous intents; no payload-level error occurs, validating the effectiveness of the schema-grounded Phase 2 mechanism.

The six reliability metrics (IWSR, SCR, SVR, HR, SS, RRF) constitute a reusable, domain-specific evaluation framework applicable to any intent-to-API orchestration system, regardless of LLM provider or underlying telecom infrastructure.

Future work includes: cross-model reliability benchmarking (Gemini, GPT-4o, open-weight models) to establish provider-agnostic bounds; integration of 3GPP ZSM closed-loop control for fully autonomous network management; user studies measuring the impact of intent phrasing on translation reliability; extension of the evaluation dataset with real operator intent logs; and exploration of fine-tuned smaller models to reduce per-intent LLM cost while maintaining reliability targets.

All source code, the ground-truth dataset, and evaluation scripts are publicly available at: <https://github.com/doadurak/mini-telco-platform>.

REFERENCES

- [1] 3GPP, “System Architecture for the Common API Framework for 3GPP Northbound APIs,” 3GPP TS 23.222, 2023.
- [2] J. Ordóñez-Lucena and C. Dsouza, “Pathways Towards Network-as-a-Service: The CAMARA Project,” *ACM SIGCOMM Workshop on Network-Application Integration*, 2022.
- [3] A. Molina Sanchez et al., “Offering the 3GPP Common API Framework as Microservice to Vertical Industries,” *EuCNC/6G Summit*, IEEE, 2022.
- [4] D. Fragkos et al., “NEFSim: An Open Experimentation Framework Utilizing 3GPP’s Exposure Services,” *EuCNC/6G Summit*, IEEE, 2022.
- [5] B. Turkovic et al., “Dynamic QoS Adaptation via Exposed Service APIs for Advanced CAM Use Cases,” *EuCNC/6G Summit*, IEEE, 2025.
- [6] A. Maatouk et al., “Large Language Models for Telecom: Forthcoming Impact on the Industry,” *IEEE Communications Magazine*, vol. 63, no. 1, pp. 62–68, Jan. 2025.
- [7] S. Lee et al., “TelAgentBench: A Multi-faceted Benchmark for Evaluating LLM-based Agents in Telecommunications,” *EMNLP 2025 Industry Track*.

- [8] Z. Zeng et al., “FlowMind: Automatic Workflow Generation with LLMs,” *ACM ICAIF*, 2023.
- [9] Y. Liu et al., “Are LLMs Good at Structured Outputs?,” *Information Processing and Management*, vol. 61, Art. 103809, 2024.
- [10] A. Rafiq et al., “Intent-Based End-to-End Network Service Orchestration System for Multi-Platforms,” *Sustainability*, vol. 12, no. 7, p. 2782, 2020.
- [11] E. Coronado et al., “Zero Touch Management: A Survey of Network Automation Solutions for 5G and 6G Networks,” *IEEE Commun. Surveys Tuts.*, 2022.
- [12] N. F. Saraiva de Sousa et al., “Network Service Orchestration: A Survey,” *Computer Communications*, vol. 142–143, pp. 69–94, 2019.